IEEE-CIS Fraud Detection

Document 3 of 3 — Modeling, Cascade Architecture & Evaluation Metrics

---

This document covers everything from model training through to the final cascade evaluation: which models were used and why, how out-of-fold (OOF) predictions work, the 5-way stacking comparison, why ROC-AUC and PR-AUC were chosen as the primary metrics, the full cascade architecture and why it's structured the way it is, and — since it comes up constantly — **exactly how the 3.5% fraud imbalance was and wasn't handled** at every stage.

Notebook sections covered: Section 5 (Level 1 — three base models), Section 6 (Level 2 — the stack), and Section 6.2/6.3 (the true cascade and its results). All numbers in this document are the actual, real outputs from running the notebook on the full IEEE-CIS dataset — nothing here is invented.

1. Why three different models instead of one

The notebook trains three gradient-boosted tree models — **LightGBM**, **XGBoost**, and **CatBoost** — on the same 427-feature matrix built in Document 2, rather than picking one and calling it done.

Why this design choice

The three libraries grow trees in genuinely different ways: LightGBM grows **leaf-wise** (always splitting whichever leaf reduces error the most, regardless of tree shape), XGBoost grows **depth-wise** (level by level, more balanced trees), and CatBoost uses **oblivious trees with ordered boosting** (a technique specifically designed to reduce a certain kind of target leakage that can creep into boosting on categorical-heavy data). Because they make genuinely different kinds of mistakes on the same rows, their errors are only partially correlated — which is exactly what makes combining them (Section 3, "stacking") worth more than just training one model for three times as long.

1.1 How each model was configured

Model	Key settings used	What they control
LightGBM	num_leaves=256, learning_rate=0.08, feature_fraction=0.30, reg_alpha=0.30, reg_lambda=0.60	Leaf-wise growth needs regularization (feature sub-sampling + L1/L2) to avoid overfitting the 427-feature matrix
XGBoost	max_depth=12, subsample=0.80, colsample_bytree=0.40, min_child_weight=4	Deep but heavily feature-subsampled trees; min_child_weight guards against splits on tiny, unreliable groups
CatBoost	depth=8, l2_leaf_reg=3.0, bootstrap_type=Bernoulli, subsample=0.80	Shallower trees (CatBoost's oblivious-tree structure is more expressive per level); Bernoulli subsampling for regularization

All three share the same `learning_rate` and `SEED=42` for a fair, reproducible comparison, and all three use **early stopping** (150 rounds without validation-AUC improvement) so training halts automatically at the

right point per fold rather than running a fixed number of rounds regardless of overfitting.

2. How training + out-of-fold (OOF) prediction actually works

This is the mechanism underneath every number in this document, so it's worth being precise about it. Recall from Document 1, Section 4: the train set is split into 3 time-grouped folds. For **each** of the three models, the training loop does this **three separate times** (once per fold):

- Train the model on 2 of the 3 folds' data (the "training" months for that round).
- Predict on the 1 held-out fold (months the model never trained on) — these predictions are stored into an `oof` array, at exactly the row positions of that held-out fold.
- Also predict on the real, unlabeled competition test set, and average that prediction across all 3 folds' models — this becomes the final `test_pred` for that model.

End result after 3 rounds: every single training row now has exactly one OOF prediction, made by a model that never saw that row (or its month) during training. This full-length array — one prediction per training row, zero leakage — is what every AUC/precision/recall number for the base models and the stack is computed from. It is the closest honest stand-in for "how will this model do on the real, future test set" that's available without having the actual test labels.

Actual output when run (excerpt — LightGBM)

```
[lgb] fold 0  auc=0.93677  best_iter=379  (190s)
[lgb] fold 1  auc=0.94707  best_iter=343  (169s)
[lgb] fold 2  auc=0.95046  best_iter=448  (201s)
LightGBM OOF AUC = 0.94431
```

Notice the per-fold AUCs vary (0.937 → 0.947 → 0.950) — this reflects genuine month-to-month variation in how learnable fraud is in that period, not instability in the model. The pooled OOF AUC (0.94431) is computed from all three folds' held-out predictions combined into one array against the true labels.

2.1 All three base models — final OOF results

Model	OOF ROC-AUC	Training time (all 3 folds)
LightGBM	0.94431	561.3s
XGBoost	0.94332	653.9s
CatBoost	0.93559	5,023.4s (~84 min)

LightGBM is both the fastest and the highest-scoring individual model here — this is exactly why it was chosen as the Stage-1 screening model in the cascade (Section 6): it's the best available combination of speed and standalone accuracy.

2.2 Are the three models actually disagreeing? (Correlation check)

Before combining models, it's worth checking whether they're worth combining at all — if two models always agree, averaging them adds nothing.

	lgb	xgb	cat

lgb	1.0000	0.9556	0.9467
xgb	0.9556	1.0000	0.9524
cat	0.9467	0.9524	1.0000

Correlations sit around 0.95, comfortably in the "genuine disagreement, worth stacking" range (roughly <0.98) rather than "near-duplicate, blending won't help" (roughly >0.995). This justifies building the ensemble in Section 3.

3. Stacking / ensembling — combining the three models

Five different combination strategies were built and compared, all evaluated the same, leakage-safe way: **nested** — for every fold, the combiner itself is fit only on the *other* folds' OOF predictions, then asked to combine that held-out fold's predictions. Fitting a combiner on the full OOF vector and reporting that score would silently overfit the combining step itself; this notebook avoids that.

Combiner	How it works
mean_prob	Plain average of the 3 models' probabilities. No fitted parameters — nothing to overfit.
rank_mean	Average of each model's within-fold rank, not raw probability — immune to models being differently calibrated (e.g. one model's "0.3" not meaning the same risk level as another's).
weighted_rank	Rank average, but weights are searched over a grid to maximize AUC per fold.
logit_lr	Logistic regression fit on the logit-transformed probabilities — learns how much to trust each model.
lgb_stack	A small second-level LightGBM trained on the 3 models' logits plus the top 12 raw features by importance ("restacking") — the most flexible combiner.

Actual output when run — full scoreboard

```
combiner         auc_oof   auc_fold_mean   auc_fold_std   level
weighted_rank    0.94773   0.94782         0.00487        stack
mean_prob        0.94747   0.94754         0.00420        stack
logit_lr         0.94725   0.94833         0.00507        stack
rank_mean        0.94716   0.94733         0.00467        stack
lgb_stack        0.94523   0.94810         0.00511        stack
lgb              0.94431   0.94477         0.00582        base
xgb              0.94332   0.94342         0.00471        base
cat              0.93559   0.93582         0.00453        base

selected: weighted_rank   (auc_oof=0.94773)
best single model: lgb   stack gain: +0.00342 AUC
```

Every stacked combiner beat every single base model — confirming the 3-model approach was worth the extra training cost. The gain is modest (+0.0034 AUC over LightGBM alone) but real and consistent, which is typical for well-built ensembles: most of the score already came from feature engineering (Document 2), and stacking picks up the remaining, harder-to-capture disagreement between models.

`auc_fold_mean` and `auc_fold_std` are reported alongside the pooled `auc_oof` deliberately: the pooled number can look deceptively stable even if performance actually swings a lot fold-to-fold. Here the std (~0.004-0.005) is small relative to the gaps between combiners, so the ranking above is trustworthy, not noise.

4. Evaluation metrics — what they are and why these were chosen

This section answers the question a judge is most likely to ask: **why these specific metrics, and not plain accuracy?** The short answer, tying back to Document 1: fraud is only 3.499% of the data, and accuracy is badly misleading on imbalanced data.

Concrete illustration of why accuracy fails here: a model that predicts "not fraud" for every single transaction, with zero actual fraud-detection ability, would score **96.5% accuracy** on this dataset — because 96.5% of rows really are legitimate. Accuracy cannot tell that model apart from a genuinely useful one. This is exactly why it is not used anywhere as a primary metric in this project.

4.1 ROC-AUC (Receiver Operating Characteristic — Area Under Curve)

What it measures: if you picked one random fraud case and one random legitimate case, ROC-AUC is the probability the model scores the fraud case higher. It ranges 0.5 (no better than random guessing) to 1.0 (perfect separation).

Why it's imbalance-resistant: ROC-AUC is *threshold-free* and *rank-based* — it never asks "how many did you classify correctly", only "did you rank the true frauds above the true non-frauds, on average". A model that never predicts fraud scores 0.5 (random) on this metric, not 96.5% — it can't hide behind the class imbalance the way accuracy lets it.

4.2 PR-AUC (Precision-Recall — Area Under Curve, a.k.a. Average Precision)

What it measures: the area under the precision-recall curve — how well precision holds up as recall increases, across every possible decision threshold.

Why it's used *in addition to* ROC-AUC, not instead of it: ROC-AUC's denominator includes the huge number of true negatives (565,000+ legitimate transactions), which can make it look artificially healthy even when a model is actually quite bad at the specific, hard problem of ranking rare positives highly. PR-AUC's baseline is the positive rate itself (3.5% here — a random model scores ~0.035 PR-AUC, not 0.5), so it's a much more honest, imbalance-sensitive measure of "can this model actually find the needles in the haystack". Reporting both gives a complete picture; PR-AUC in particular is the more trustworthy number to cite when explaining performance on this specific imbalanced dataset.

4.3 Precision, Recall, F1, and the Confusion Matrix

ROC-AUC and PR-AUC are both *threshold-free* — they evaluate the model's ranking ability across every possible cutoff at once. But a real deployed system has to make an actual yes/no call on each transaction, which means picking one specific threshold. Precision, recall, F1, and the confusion matrix are what tell you how the model behaves *at that one chosen operating point*.

Term	Plain-English meaning	Formula
Precision	Of everything the model flagged as fraud, what fraction actually was fraud?	TP / (TP + FP)
Recall	Of all the real fraud that existed, what fraction did the model catch?	TP / (TP + FN)

F1	A single number balancing precision and recall (their harmonic mean)	2·P·R / (P + R)
Confusion Matrix	The raw counts underneath all of the above: True Negatives, False Positives, False Negatives, True Positives	—

Why fraud detection specifically cares about the precision/recall trade-off: the two error types have very different real-world costs. A **false negative** (missed fraud) directly costs money and customer trust. A **false positive** (legitimate transaction flagged) costs a review/customer-friction expense, but nothing was actually stolen. Because these costs are asymmetric, it's not enough to know "the model is 94% accurate at ranking" — you need to know, at the threshold you'll actually deploy, how many real frauds slip through and how many innocent customers get annoyed. This is exactly why the cost-analysis section (Section 7 below) exists: it makes that asymmetry an explicit, adjustable input rather than an assumption buried in the code.

4.3.1 How the decision threshold was chosen (not test data)

The classification threshold applied to compute precision/recall/F1/confusion-matrix is chosen by finding the point that **maximizes F1 on OOF predictions** — never on the real test set, since test labels aren't available and using them would be a form of leakage/cheating even if they were. This keeps the threshold choice consistent with the no-leakage principle used everywhere else in the project (Document 1, Section 4).

5. Why a cascade at all? The problem it solves

The stacked ensemble from Section 3 is the most accurate option available (0.94773 AUC), but it has a practical downside: it requires running **all three models** — including the slow ones, XGBoost and especially CatBoost (84 minutes for 3 folds alone) — on **every single transaction**, forever, in production. Most transactions are obviously legitimate; spending three expensive model calls on all of them is wasteful.

The cascade's idea: use the **fastest, still-strong** model (LightGBM — see Section 2.1, it was both fastest and most accurate individually) as a cheap first-pass filter. Only transactions that filter is unsure about get escalated to the expensive XGBoost + CatBoost combination. Transactions the filter is confident are safe skip Stage 2 entirely — a genuine computational saving, not just a different way of scoring the same predictions.

	Full ensemble (no cascade)	Cascade
Models run per transaction	All 3, always	LightGBM always; XGBoost+CatBoost only if escalated
Cost	Highest, constant	Lower on average — scales with how many transactions look risky
Best for	Maximum possible accuracy	Production systems processing millions of transactions where compute cost matters

6. Cascade architecture — how it actually works

6.1 Stage 1: LightGBM as a high-recall screener

Every transaction, without exception, is scored once by LightGBM. This produces a risk score between 0 and 1 for every row. A **threshold** is then applied: scores at or above the threshold are "escalated" to Stage 2; everything below is immediately decided as low-risk and given a final score of 0, with no further model calls.

6.1.1 How the threshold was chosen — TARGET_STAGE1_RECALL = 98%

Rather than picking a threshold that just maximizes some overall score, the threshold is chosen with an explicit business constraint: **retain at least 98% of all known fraud** at Stage 1. The code scans 2,001 candidate thresholds (evenly spaced quantiles of the LightGBM OOF scores), computes what fraud recall each one would achieve, keeps only thresholds that clear 98% recall, and — among those — picks the *highest* such threshold (i.e. the most selective one that still satisfies the recall requirement, which in turn maximizes how many transactions can be safely filtered out).

Why 98%, specifically, and why is this framed as a business decision rather than a pure ML one: this directly reflects the class-imbalance reality from Document 1 — fraud is rare (3.5%) and expensive to miss, so Stage 1's job is deliberately *not* to be precise. It is allowed, on purpose, to flag a lot of false alarms, as long as it almost never lets a real fraud case through undetected. This is the correct way to think about a screening stage in a high-recall system: optimize the cheap stage for "don't lose the needle", and let the expensive, more accurate stage handle the actual precision work. `TARGET_STAGE1_RECALL` is a configurable variable

specifically so this business trade-off (recall target vs. how much compute gets saved) can be adjusted without touching any modeling code.

6.1.2 No leakage in threshold selection

The threshold is selected using only LightGBM's OOF predictions (Section 2) and the real training labels — both come exclusively from the training set, never the test set. This preserves the same no-leakage discipline used for the fold split and every other decision in this project (Document 1, Section 4).

6.2 Stage 2: XGBoost + CatBoost, only for escalated transactions

Only transactions that passed Stage 1's threshold are scored by XGBoost and CatBoost, and their predictions are simply averaged 50/50 to produce the final risk score. Transactions that Stage 1 filtered out never reach this code path at all — `xgb.DMatrix` and `predict_proba` are only ever called on the filtered-down subset, not the full dataset with results discarded afterward. This is what makes it a *true* computational cascade rather than just a different way of presenting the same predictions.

6.3 Cascade results — what actually happened when it ran

Actual output when run

```
Stage 1 model              : LightGBM
Stage 1 target recall       : 98.00%
Stage 1 threshold           : 0.00025694
Stage 1 OOF fraud recall    : 98.0061%
Stage 1 precision (OOF)     : 6.1677%
Stage 1 false negatives     : 412 (1.9939% of all fraud)

OOF Stage-2 workload        : 55.60%
Test Stage-2 workload       : 64.88%
Test workload reduction     : 35.12%

Cascade OOF ROC-AUC         : 0.94409
Cascade OOF PR-AUC          : 0.69078
Cascade precision (OOF)     : 0.7485
Cascade recall (OOF)        : 0.5758
Cascade F1 (OOF)            : 0.6509
Confusion matrix [tn fp; fn tp]:
[[565879   3998]
 [  8765  11898]]

Full ensemble ROC-AUC       : 0.94535
Full ensemble PR-AUC        : 0.69101

Full ensemble inference     : 76.8193 s
True cascade inference      : 73.0484 s
Measured time reduction     : 4.91%
```

6.3.1 Reading these numbers correctly

- **98.0061% Stage-1 recall** — the target was hit almost exactly. Only 412 of the ~20,663 real frauds in the training data were missed entirely by the screening stage.

- **6.17% Stage-1 precision** — expected and intentional, not a flaw. At a 98%-recall target, the threshold has to be generous, so most of what gets escalated to Stage 2 turns out to be legitimate. Stage 1's job was never to be precise (see Section 6.1.1).
- **35.12% workload reduction (test set)** — the real headline number: over a third of transactions never touched the expensive XGBoost/CatBoost models at all. This is the exact number to lead with when explaining the value of the cascade.
- **Cascade AUC (0.94409) vs. Full ensemble AUC (0.94535)** — the cascade is only 0.00126 AUC lower (and PR-AUC only 0.00023 lower) than running all three models unconditionally on every transaction. This is the core trade-off result: *~35% less Stage-2 compute for a nearly unmeasurable accuracy cost*.
- **Measured time reduction (4.91%) is much smaller than the 35.12% workload reduction** — and the notebook deliberately reports both, rather than only the flattering one. LightGBM's own Stage-1 pass still runs on 100% of rows, and real hardware doesn't speed up linearly just because there's less data in Stage 2 — fixed overhead (memory allocation, model-loading, batching) doesn't shrink proportionally. This distinction — *architectural/row-count saving* vs. *actual measured wall-clock saving* — is an important, honest nuance to be ready to explain if asked.

7. Cost analysis — turning the precision/recall trade-off into a business decision

Precision and recall alone don't say whether a given threshold is actually a *good business decision* — that depends on how costly a missed fraud is relative to how costly a false alarm is, which is not a fixed, universal number. The cost-analysis section makes this trade-off explicit and adjustable rather than hard-coded.

Four configurable, illustrative cost variables are defined:

Variable	Meaning	Notebook default
FN_COST	Relative cost of missing a real fraud case	50 (units, not real dollars)
FP_COST	Relative cost of a false alarm / unnecessary review	1 (unit)
STAGE1_COST	Relative inference cost per row for Stage 1	1 (unit)
STAGE2_COST	Relative inference cost per row for Stage 2 (only escalated rows)	20 (units)

Important: these are deliberately abstract relative-cost *assumptions*, not researched real-world dollar figures. Presenting them as actual dollar costs would be misleading — the honest framing is "here is how the optimal threshold shifts if fraud is assumed to be roughly 50x costlier than a false alarm", not "missing a fraud costs exactly $50".

The notebook sweeps every candidate Stage-1 threshold, computes a total cost (`FN_count × FN_COST + FP_count × FP_COST + inference costs`) for each, and finds the threshold that minimizes total cost — then also runs a sensitivity sweep (what if fraud is 4x costlier? what if reviews are 4x costlier? what if Stage 2 gets 4x cheaper?) to show how much the optimal threshold moves under different business assumptions. This turns "what threshold should we deploy" from a single fixed number into an explicit, presentable trade-off that a business stakeholder could actually weigh in on.

8. How the 3.5% class imbalance was actually handled (direct answer)

This is worth spelling out clearly and completely, since it's a natural question for anyone reviewing this project: **this notebook does not use SMOTE, oversampling, undersampling, or class-weighting anywhere.** That is a deliberate choice, not an oversight, and here is exactly how imbalance was addressed instead, at every stage:

Stage	How imbalance was actually addressed
Metric choice (Section 4)	ROC-AUC and PR-AUC were used instead of accuracy specifically because they are not distorted by class imbalance — a rank-based metric doesn't reward a model for exploiting the majority class.

Validation (Document 1, Sec. 4)	Both a pooled OOF score and per-fold mean/std are reported, since a rare positive class means fold-to-fold fraud rate genuinely varies (2.89% to 3.98% across the 3 folds seen in Document 1) — a single pooled number can hide that variation.
Model training	No class_weight, scale_pos_weight, or is_unbalance parameter is set on any of the three models. Tree-based models tolerate class imbalance reasonably well on their own, especially when evaluated with a threshold-free, rank-based metric.
Cascade design (Section 6)	This is the main place imbalance genuinely shaped the architecture: TARGET_STAGE1_RECALL=98% exists specifically because fraud is rare and costly to miss — the cascade deliberately over-catches at Stage 1 (accepting low 6% precision) rather than risk losing rare fraud cases to a stricter, more "balanced-looking" threshold.
Cost analysis (Section 7)	FN_COST is set 50x higher than FP_COST by default — a direct, explicit acknowledgment that the minority class's errors matter far more than the majority class's, without needing to touch the training data at all.

8.1 Why SMOTE/oversampling was deliberately skipped (worth saying out loud)

- **SMOTE generates synthetic minority examples by interpolating between real ones.** For tree-based models — which split on real, raw feature thresholds — synthetic interpolated points often land in unrealistic regions of feature space and mostly add noise rather than signal; this is a well-documented limitation, not a guess.
- **Tree ensembles don't need balanced classes to split effectively** the way some other model families (e.g. plain logistic regression) sometimes do — they can naturally learn asymmetric decision boundaries.
- **Rebalancing distorts calibration** — after SMOTE or class-weighting, a model's raw output probability no longer reflects the true underlying fraud rate, which would complicate both the OOF-based threshold selection in Section 6.1.1 and the cost analysis in Section 7, both of which rely on scores being comparable across the true, un-rebalanced class distribution.

The one-sentence version to say out loud if asked: "We didn't rebalance the training data — tree models handle imbalance reasonably well on their own, and we optimized directly for AUC/PR-AUC, which aren't distorted by imbalance the way accuracy is. Instead, we addressed the *cost* of imbalance directly: Stage 1's recall target is set to 98% specifically because fraud is rare and expensive to miss, even though that means accepting a lot of false alarms — that trade-off is quantified explicitly in the cost-analysis section."

9. Summary — the complete modeling story, end to end

- **Three different tree models** (LightGBM, XGBoost, CatBoost) were trained because their differing inductive biases produce only partially correlated errors (~0.95 correlation) — genuinely worth combining.
- **Out-of-fold (OOF) prediction**, built from the time-aware 3-fold split (Document 1), ensures every reported metric reflects a model predicting on data it never trained on — the honest proxy for real future performance.

- **Stacking 5 ways** confirmed every combiner beats every single base model; `weighted_rank` won at 0.94773 AUC.
- **ROC-AUC and PR-AUC** were chosen over accuracy specifically because the 3.499% fraud rate would let a useless model score 96.5% accuracy — these two metrics don't have that flaw.
- **The cascade** uses LightGBM (fastest + most accurate single model) as a 98%-recall screening gate, sending only escalated transactions to the expensive XGBoost+CatBoost stage — a genuine, verified 35.12% reduction in Stage-2 compute, at a cost of just 0.00126 AUC versus the full ensemble.
- **Precision/recall/F1/confusion-matrix** were computed at an OOF-selected decision threshold to show real-world deployment behavior, not just abstract ranking ability.
- **A cost-analysis layer** makes the recall-vs-precision trade-off an explicit, adjustable business decision rather than a value buried in code.
- **Class imbalance (3.5% fraud)** was handled through metric choice, validation design, and the cascade's recall-first Stage-1 target — not through SMOTE, oversampling, or class weights, a deliberate and defensible choice explained fully in Section 8.

End of Document 3 — this completes the full EDA → Feature Engineering → Modeling documentation set.